

STL Analyzer

A beginner-friendly guide to the Rust parser, mesh welding, quality checks, generators, tests, performance, and safe modification points.

Project: stl-analyzer | Language: stable Rust | Dependencies: none | Guide generated 3 October 2026

Progress Update

What works: strict ASCII STL parsing, structurally detected binary STL parsing, exact duplicate-vertex removal, indexed triangles, bounding boxes, sliver/large-triangle/high-incidence warnings, console menu, configurable sample generator, and a separate dimensioned generator binary.

| Verification | Result

| Unit and integration tests | 21 passed. Coverage includes ASCII and binary parsing, malformed files, non-finite values, deduplication/index relationships, bounds, slivers, large triangles, edge incidence, generators, and menu flow.

| Release benchmark | 1,000,000 binary STL triangles: parsing 34.673 ms, deduplication 170.974 ms, analysis 102.533 ms, total 308.180 ms.

| Performance target | Met. 308.180 ms is below the 0.86 second target on this machine.

| Estimated indexed mesh storage | 28.89 MiB. This estimate covers the final indexed vertices and triangle indices, not temporary parser/hash/sorting allocations.

Current limitations: ASCII input is intentionally strict; welding is exact rather than tolerance based; input normals are stored but not checked against geometric normals; the quality defaults are generic review heuristics, not manufacturing tolerances.

Quick Start

cargo test

cargo run --release --bin stl-analyzer

cargo run --release --bin stl-analyzer -- benchmark 1000000 benchmark_1000000.stl

Dedicated dimensioned generator

cargo run --release --bin stl-generator -- cube --size 40 --density 16

cargo run --release --bin stl-generator -- cylinder --radius 12.5 --height 80

cargo run --release --bin stl-generator -- torus --major-radius 30 --minor-radius 8 --format ascii

For distribution, put config.json next to stl-analyzer.exe and stl-generator.exe. The stl_folder setting controls the default model folder. During cargo run, the project-root config is a development fallback.

Project Map

| File | Responsibility | Change here when...

| src/main.rs | Starts the analyzer program and delegates to app::run_command. | You need a different analyzer startup behavior.

| src/app.rs | Reads config, renders the menu/report, lists STL files, runs benchmark commands. | You change prompts, report text, config fields, or command behavior.

| src/stl.rs | Detects format, parses/writes STL, reports recoverable StlErrors. | You support more STL variants or change validation rules.

| src/geometry.rs | Vec3, raw Triangle, indexed Mesh, exact welding. | You change mesh representation or vertex matching.

| src/analysis.rs | Bounds and quality findings. | You add or revise a mesh-quality rule.

| src/samples.rs | Primitive shape generation, dimensions, streaming benchmark fixture. | You add a shape or dimension parameter.

| src/bin/stl_generator.rs | Separate command-line dimensioned generator. | You add generator flags or output options.

| tests/ and module tests | Regression checks for behavior. | You change expected behavior. Update/add tests before refactoring.

End-to-End Data Flow

STL bytes from disk

-> stl::detect_format

```
-> stl::parse_ascii or stl::parse_binary
-> Vec<Triangle> (three coordinates per face)
-> Mesh::from_triangles (welds exact duplicate vertices)
-> indexed Mesh { vertices, triangles: indices }
-> analysis::analyze
-> console report or benchmark timings
```

Useful Rust Concepts in This Project

- struct groups related data: for example, Triangle contains a normal and exactly three Vec3 vertices.
- enum expresses alternatives: StlFormat::Ascii or StlFormat::Binary; StlError::Io or StlError::Invalid.
- Result<T, E> represents successful work or a recoverable error. Parser callers use ? to return an error without a panic.
- &[T] borrows a slice without copying it. Writers receive &[Triangle].
- Vec<T> owns a growable list. The parser owns its triangles; welding consumes that vector and builds the indexed mesh.

Parsing STL Files

Binary STL

A valid binary STL is exactly: 80-byte header, 4-byte little-endian triangle count, then 50 bytes per triangle. The parser first checks whether $84 + 50 * \text{count} == \text{file length}$. That structural check is important because a binary header is allowed to start with the text solid.

Each triangle contains 12 floats: three normal components, then three vertices with three components each; the final 2 bytes are an unused attribute count. Non-finite coordinates such as NaN or infinity are rejected.

ASCII STL

The parser requires a UTF-8 document beginning with solid, followed by repeated complete facets:

```
facet normal nx ny nz
outer loop
vertex x y z
vertex x y z
vertex x y z
endloop
endfacet
endsolid name
```

Being strict produces helpful errors for malformed files. If your mentor wants to accept exporters that omit normals, vary whitespace unusually, or add extra comments, adjust parse_ascii in src/stl.rs and add a regression test first.

Why the Parser Returns Raw Triangles First

STL repeats every vertex for every triangle. A raw triangle list matches the file directly, so parsing is simple. The next stage can then choose the mesh identity policy (currently exact welding) without making format code responsible for mesh topology.

Welding and Indexed Meshes

Mesh::from_triangles walks every raw vertex once. A HashMap<VertexKey, usize> maps an exact coordinate key to the index of the first matching vertex in mesh.vertices. Each output face stores only three usize indices in IndexedTriangle.

Raw faces: [A, B, C] [B, D, C]

Unique vertices: [A, B, C, D]

Indexed faces: [0, 1, 2] [1, 3, 2]

Important: this is exact welding. The key uses f32::to_bits(), except -0.0 is normalized to 0.0. Coordinates that differ by even a tiny amount remain separate. That avoids accidentally modifying model geometry, but can leave seams in imperfect exports.

Analysis Rules

Bounding Box

`BoundingBox::from_vertices` visits each unique vertex and tracks component-wise minimum and maximum X, Y, Z. It returns None for an empty mesh. The squared diagonal is used as a scale-aware area reference.

Sliver Triangles

For each indexed face, the code calculates area from the cross product and finds the longest edge. The aspect ratio is:

$$\text{aspect_ratio} = \text{longest_edge}^2 / (2 * \text{area})$$

A degenerate triangle has zero area and therefore receives infinite aspect ratio. The default warning threshold is 20.0; a larger ratio means a thinner triangle.

Very Large Triangles

The relative-area heuristic is:

$$\text{relative_area} = \text{triangle_area} / \text{bounding_box_diagonal_squared}$$

At the default 0.25, a triangle occupying at least one quarter of that reference area is reported. This is scale invariant, but it is not a standard STL rule: a flat or very elongated model may make the result surprising.

High-Incidence Vertices

Each face contributes its three undirected indexed edges. Self-edges from degenerate faces are ignored. Edges are sorted and deduplicated, then each endpoint degree is counted. A vertex is reported when its unique incident-edge count is greater than the configured default of 12.

Sorting a compact `Vec<(usize, usize)>` is intentional. It avoids a separate hash set for every vertex while still preventing the two faces on a manifold shared edge from being counted twice.

Configuration

```
{
"stl_folder": "stl_files",
"sliver_aspect_ratio": 20.0,
"large_triangle_area_ratio": 0.25,
"max_incident_edges": 12
}
```

All three threshold values must be positive. Keep them in version control with an explanation from the team, because they define what the report calls a warning.

Generators

`stl-generator` is independent from the analyzer menu. It defaults to binary output, density 24, and the configured STL folder.

Run `cargo run --release --bin stl-generator` with no arguments for an interactive flow: choose Cube, Cylinder, or Torus, then press Enter to accept each dimension default.

```
| Shape | Dimension flags | Defaults
| Prism | --width --height --depth | 2 x 1 x 1
| Cube | --size | 1
| Cylinder | --radius --height | 1, 2
| Cone | --radius --height | 1, 2
| Sphere | --radius | 1
| Torus | --major-radius --minor-radius | 1, 0.35
```

All dimensions must be finite and positive. For a torus, major radius must be greater than minor radius to avoid a self-intersecting default surface. Density is a mesh-resolution setting, not a physical dimension.

Tests and Safe Change Workflow

- Read the relevant module and its nearby tests before editing.
- Write a failing test that describes the intended new behavior.
- Make the smallest targeted change.
- Run `cargo fmt` and `cargo test`.
- For parser or analysis performance work, rerun the release benchmark with the same fixture

and record the command, machine, and output.

| Behavior | Where it is tested
| ASCII/binary parsing and malformed input | src/stl.rs unit tests; tests/loading.rs
| Deduplication and index correctness | src/geometry.rs unit test
| Bounds and quality classification | src/analysis.rs unit tests; tests/analysis.rs
| Console flow for both encodings | src/app.rs unit test
| Generator and custom dimensions | src/samples.rs unit tests

Questions for Your Mentor

These rules are domain decisions, not facts built into STL. Ask and record the answers before changing the defaults.

- What exact sliver definition should the project use: aspect ratio, minimum angle, radius ratio, or a manufacturing-specific tolerance?
- Should zero-area triangles be reported separately as ?degenerate? instead of being included as slivers?
- What should ?very large? mean for flat, long, or thin models: area versus bounding-box diagonal squared, model surface area, maximum edge length, or an absolute unit?
- What are the real model units: millimeters, inches, or unitless coordinates? Are absolute quality thresholds needed in addition to relative ones?
- Should nearly equal vertices be welded with a tolerance? If yes, what tolerance is safe and should it be absolute, relative, or both?
- Does a high-incidence vertex mean non-manifold topology, a valid CAD feature, or simply a review warning? What threshold applies to your models?
- Should the application validate stored STL normals against recomputed geometric normals and warn about inverted winding?
- Should malformed ASCII variants be accepted for compatibility, or should strict parsing remain a deliberate quality gate?
- Do benchmark requirements include peak process memory, not only final indexed-mesh storage? If yes, choose an OS-specific measurement approach and a reference machine.

Common Changes, Safely

| Goal | Likely change | Test to add
| Add a quality rule | Add a finding type and field to Analysis; calculate it in analyze; display it in format_report. | A small mesh that should produce exactly one finding and one that should not.
| Use tolerance welding | Replace VertexKey policy carefully. Consider spatial buckets before comparing nearby coordinates. | Vertices inside/outside tolerance; preserved triangle indices; no unwanted merging at large coordinates.
| Add a primitive | Add a SampleKind variant, dimensions/defaults/validation, generation function, and generator help text. | Triangle count, finite coordinates, expected bounds, invalid dimensions.
| Relax ASCII parsing | Change only parse_ascii; keep binary structural detection first. | Representative export accepted plus malformed input still rejected.
| Reduce memory | Profile before changing data flow. A streaming parser or edge analysis redesign may help, but must preserve all reports. | Existing suite plus a benchmark comparison with the same fixture.

Final advice: stay close to the current split: parsing turns bytes into triangles, geometry turns triangles into an indexed mesh, analysis reads that mesh, and app code handles people and files. That boundary makes changes easier to reason about and easier for your mentor to review.